

Pthreads

POSIX threads: many flows of execution inside one shared process

Builds on `fork()` • `exec()` • `wait()`

ONE PROCESS

T1

own stack + registers

T2

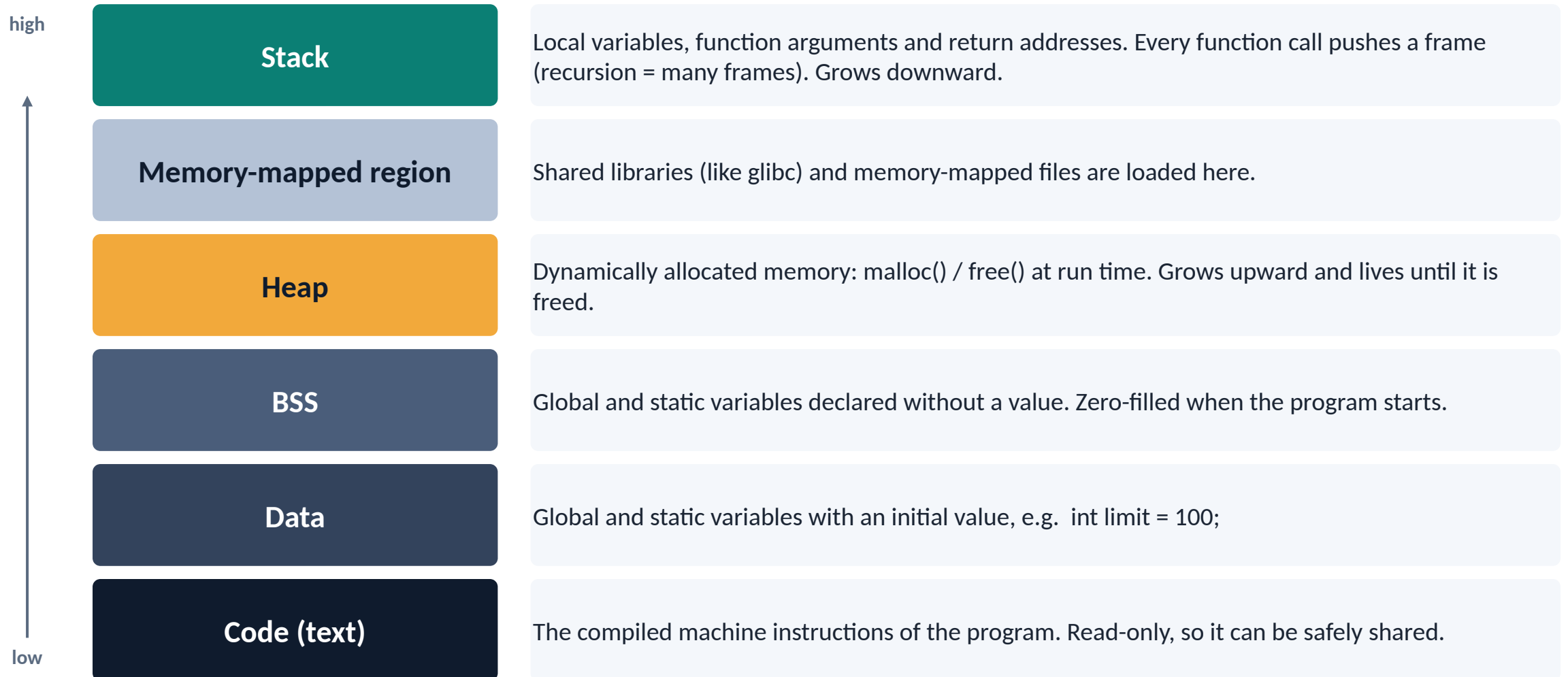
own stack + registers

T3

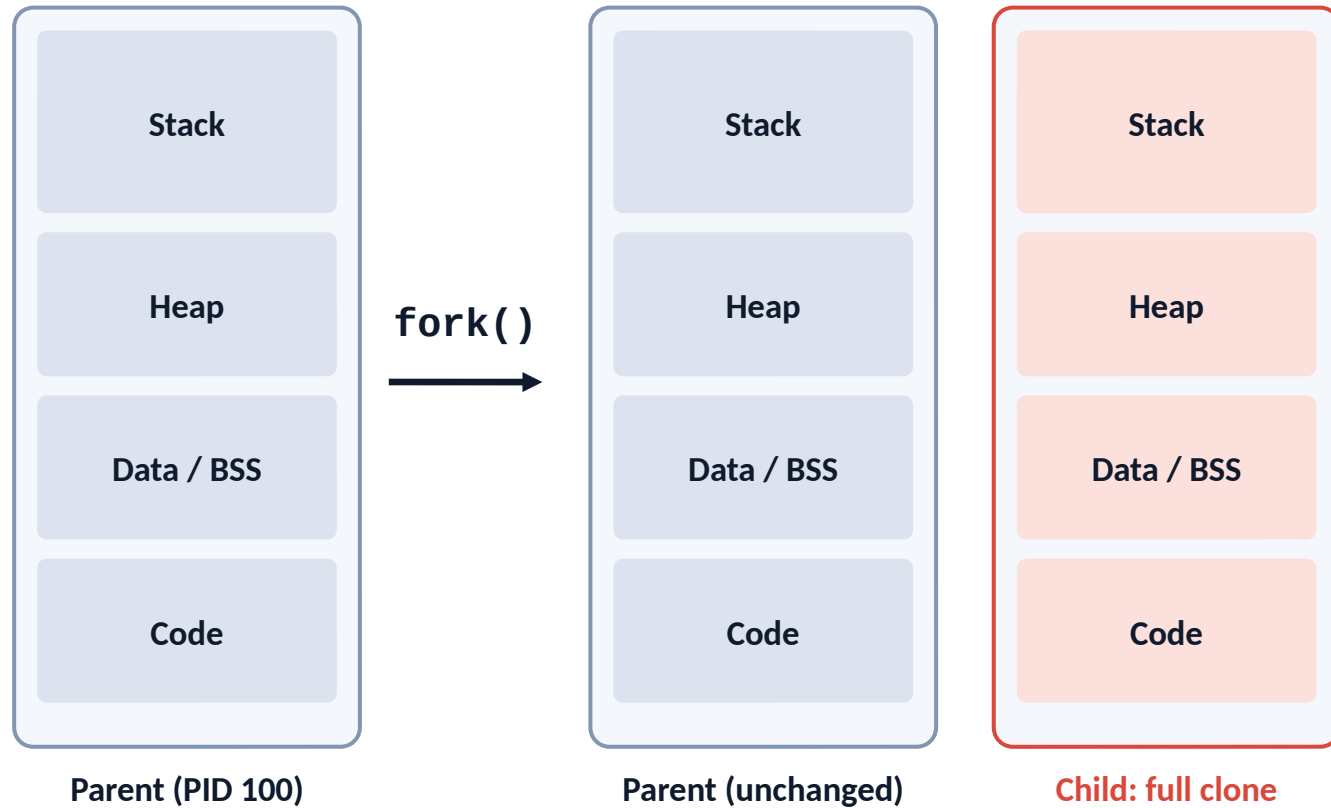
own stack + registers

shared: code • data • heap

Recall: memory space of a process



Why fork() is expensive



Two separate address spaces



Clones the whole address space

Page tables are duplicated and every segment gets its own copy (copy-on-write only postpones the copying).



Memory is isolated

Parent and child cannot see each other's variables, so data must travel through pipes or other IPC.

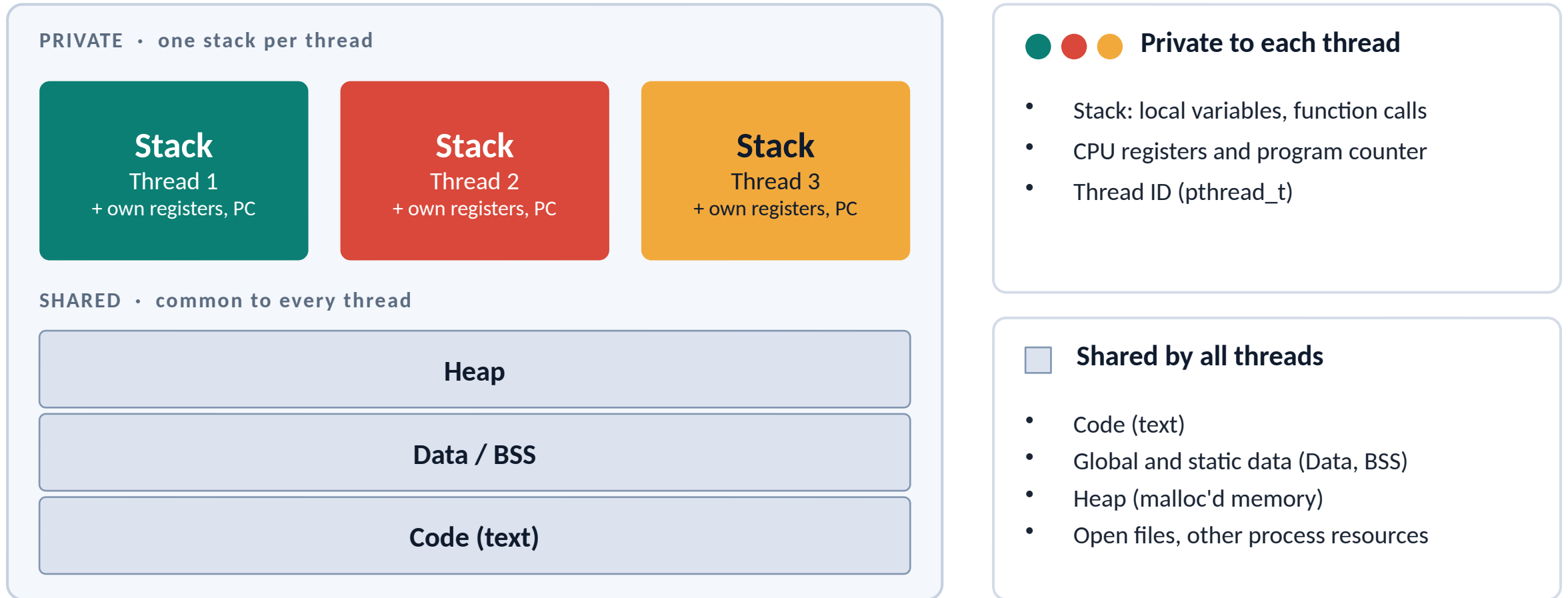


Heavier to create and switch

More kernel work than needed when all we want is concurrent work on the same data.

Idea: don't clone everything. Share what is useful, and give each flow of execution only its own stack.

Threads share everything except the stack

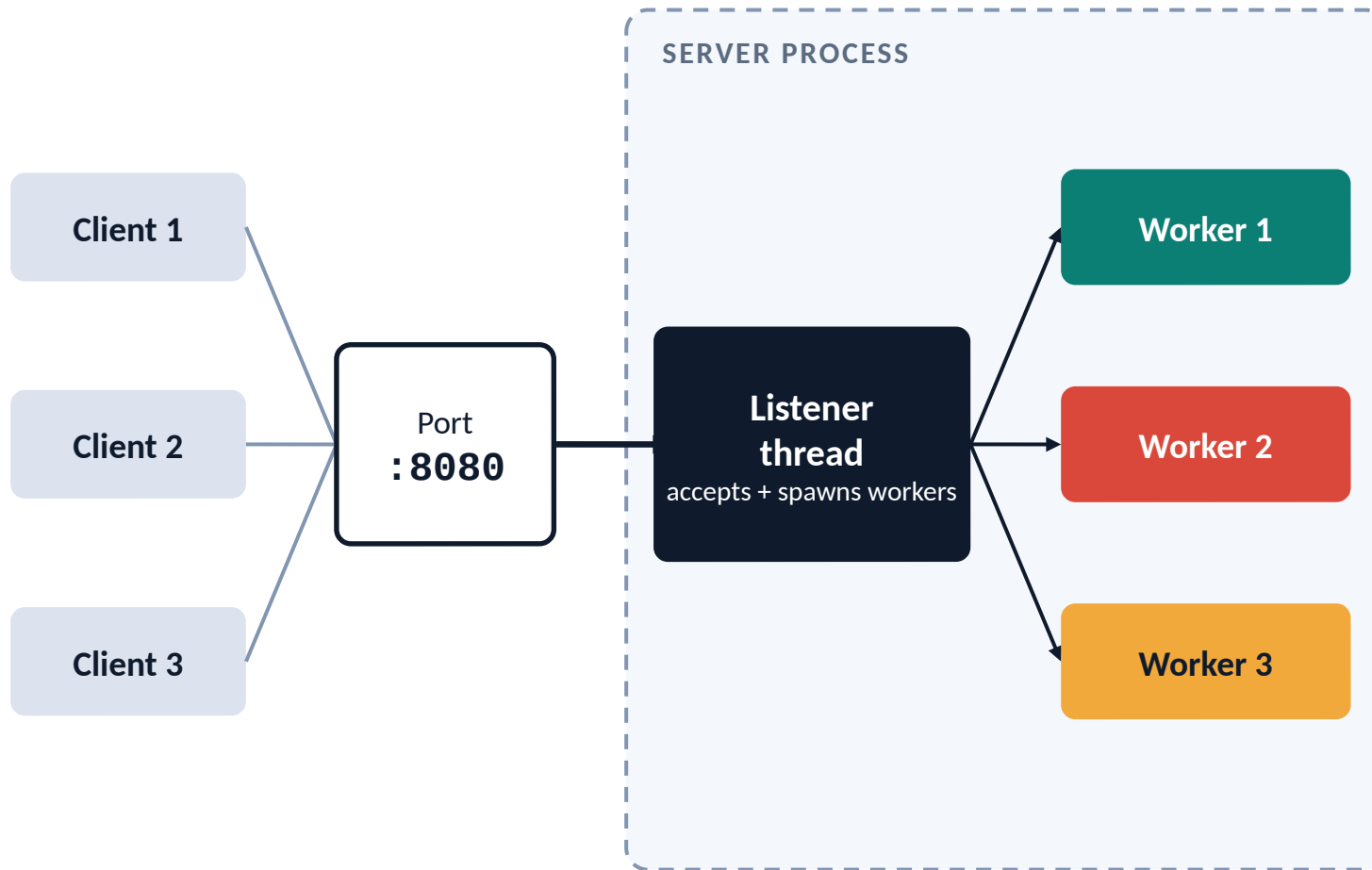


Thread: an independent execution unit

A thread is an independent flow of control with its own stack, registers and program counter. The kernel schedules it on its own, even on a different CPU core.

	Process (fork)	Thread (pthread_create)
Memory	Own private address space (a copy)	Shared with sibling threads; only the stack is private
Creation cost	Heavy: the whole memory space is cloned	Light: just a new stack and a set of registers
Communication	Needs IPC: pipes, sockets, shared memory	Simply read and write shared variables
Isolation	Strong: a crash stays inside one process	Weak: one bad thread can take down the whole process
Identity	getpid(), getppid()	pthread_self()

What are threads used for?



Listener thread

Waits on a port for new connections. It hands each request over and goes straight back to listening.

Worker threads

Each one does the actual job for one request, so slow work never blocks the listener.

Also: parallel computing on many cores, background I/O.

The Pthreads library

Your C program

ISO C standard

The core language: printf, malloc, strlen, math, ...

POSIX (Unix / Linux)

fork, exec, wait, getpid ...
and pthread_*

glibc GNU C library: implements both, incl. the Pthreads library

Linux kernel runs and schedules every thread

A POSIX standard

Same thread API on Unix-like systems.

Not part of ISO C

Available on Linux and Unix, not guaranteed elsewhere.

```
#include <pthread.h>
$ gcc prog.c -o prog -pthread
```

Core Pthreads functions

IN THE PROCESS WORLD

IN THE THREAD WORLD

`fork()`



```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,  
void *(*start_routine)(void *), void *arg);
```

Starts a new thread running `start_routine(arg)`; its ID is stored in `*thread`. Pass NULL for `attr` to use defaults.

`exit()`



```
void pthread_exit(void *retval);
```

Ends the calling thread; `retval` is passed to the joiner. A plain return does the same.

`wait() / waitpid()`



```
int pthread_join(pthread_t thread, void **retval);
```

Waits for a thread to finish and collects its return value. Like `wait()` for processes.

`getpid()`



```
pthread_t pthread_self(void);
```

Returns the ID of the calling thread.

Example: creating and joining threads

```
#include <stdio.h>
#include <pthread.h>

void *worker(void *arg) {
    int id = *(int *)arg;
    printf("Hello from thread %d\n", id);
    return NULL;
}

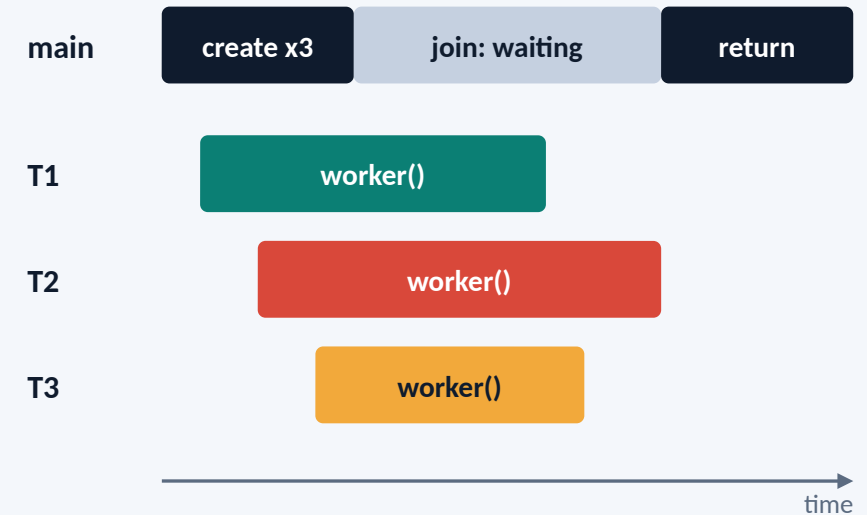
int main(void) {
    pthread_t t[3];
    int ids[3] = {1, 2, 3};

    for (int i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, worker, &ids[i]);

    for (int i = 0; i < 3; i++)
        pthread_join(t[i], NULL);

    return 0;
}
```

What happens over time



```
$ gcc hello.c -o hello -pthread
$ ./hello
Hello from thread 2
Hello from thread 1
Hello from thread 3 (order may vary)
```

Shared memory needs synchronization

	Thread A	counter (shared)	Thread B
t1	read counter → 5	5	
t2		5	read counter → 5
t3	write 5 + 1 = 6	6	
t4		6	write 5 + 1 = 6

Lost update: counter should be 7, but it is 6

Why it happens

All threads see the same globals and heap. counter++ is really load, add, store, and threads can interleave between those steps.

The fix: mutual exclusion

```
pthread_mutex_lock(&m);  
counter++;  
pthread_mutex_unlock(&m);
```

Detailed walkthrough on the board

Key takeaways

1

A thread is an execution unit

An independent flow of control inside a process, scheduled by the kernel.

2

Share everything but the stack

Code, data and heap are common. Each thread owns just its stack and registers.

3

pthread_create / exit / join

POSIX API from <pthread.h>, built with -pthread. Not part of the ISO C standard.

4

Shared memory means synchronize

Guard shared data (for example with a mutex) to avoid race conditions.